Team 8 Seminar Presentation

- Kien Mai
- Quang Anh Le
- Jack Nguyen
- Arshdeep Singh

Agenda

- Class and Functional components in React
- GitHub webhook
- Linting (Eslint) + Prettier
- Local development with Docker
- Monorepo

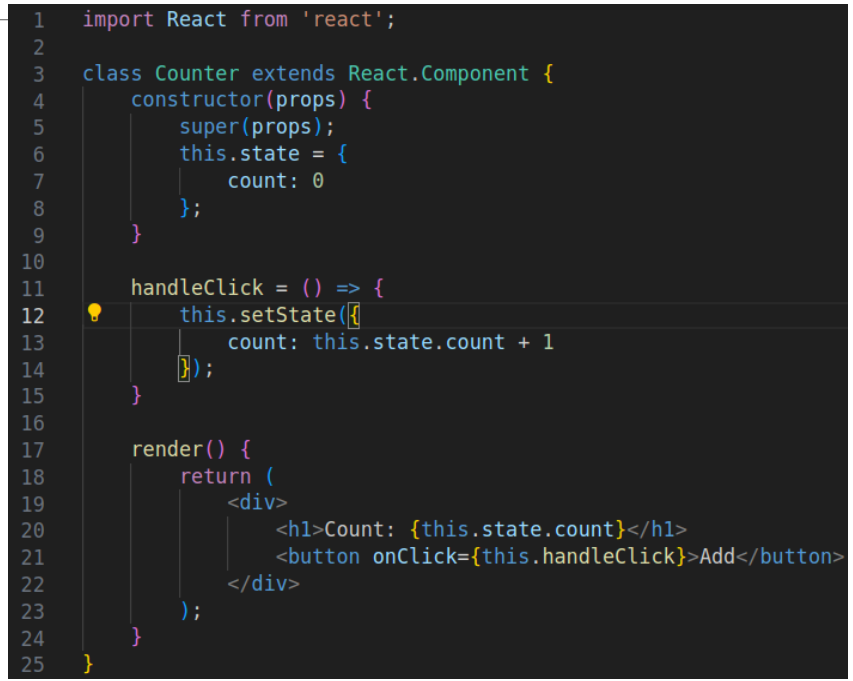
Class and Functional components in React

React Components:

- Reusable pieces of code, which can be used to build layout for web pages
- Every component has a state, which is used to update the page based on some user interaction
- How the components manage and use this state is what differentiates functional and class components

Class components

- Uses ES6 class to define a component, which extends the React.Component class
- Internal state is managed using this.state object and setState method



```
import React from 'react';

class Counter extends React.Component {
    constructor(props) {
        super(props);
        this.state = {
            count: 0
        };
    }

    handleClick = () => {
        this.setState({
            count: this.state.count + 1
        });
    }

    render() {
        return (
            <div>
                <h1>Count: {this.state.count}</h1>
                <button onClick={this.handleClick}>Add</button>
            </div>
        );
    }
}
```

Functional components

- Roughly follows the same syntax as regular JavaScript function
- The state is not stored internally in the component
- We use hooks, useState in this case, to access and modify the state of the our component

```jsx
import React, { useState } from 'react';

function Counter() {
    const [count, setCount] = useState(0);

    const handleClick = () => {
        setCount(count + 1);
    }

    return (
        <div>
            <h1>Count: {count}</h1>
            <button onClick={handleClick}>Add</button>
        </div>
    );
}
```

Advantages of functional components

- As evident from the previous slides, we can see that functional components are more readable and use fewer lines of code
- Due to this benefit, they are the default way to create components on the official React documentation website

```
import React from 'react';

class Counter extends React.Component {
    constructor(props) {
        super(props);
        this.state = {
            count: 0
        };
    }

    handleClick = () => {
        this.setState({
            count: this.state.count + 1
        });
    }

    render() {
        return (
            <div>
                <h1>Count: {this.state.count}</h1>
                <button onClick={this.handleClick}>Add</button>
            </div>
        );
```

```
import React, { useState } from 'react';

function Counter() {
    const [count, setCount] = useState(0);

    const handleClick = () => {
        setCount(count + 1);
    }

    return (
        <div>
            <h1>Count: {count}</h1>
            <button onClick={handleClick}>Add</button>
        </div>
    );
}
```

Cases for use of class components

- While functional components are the go to way of creating components, there may be cases where class components can make sense.
- For e.g. componentDidCatch, a lifecycle method which can catch errors in the children components and display a fallback UI instead of a blank screen
- Currently there is no equivalent hook for this lifecycle method, this functionality can only be implemented using class components.

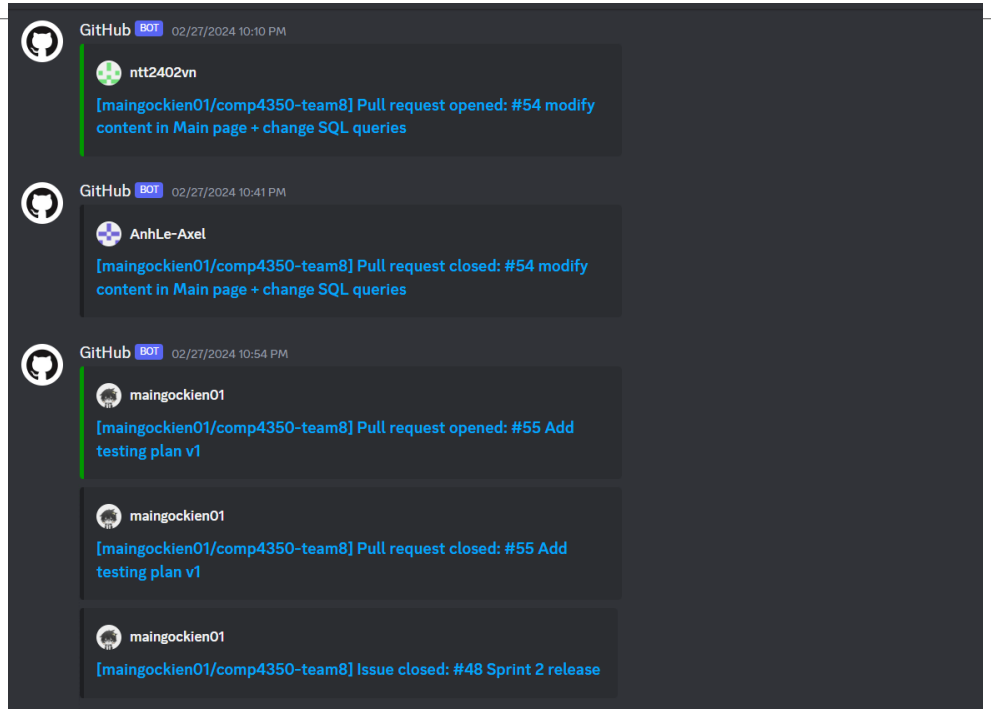
Takeway

- For simpler applications like our project, everything can be achieved through functional components and hooks
- Therefore, it makes sense to use them and write code which is easier to create and maintain

GitHub Webhook

- Notification system of Github
- Let you subscribe to events happening in a repository.
- These events can include actions such as pushes to the repository, pull requests, issue updates, and more.

GitHub Webhook (Example)



GitHub Webhook

Benefits:

- Real-time notification
- Centralized Communication instead of switching between different platforms
- Increased Awareness and Transparency of team members

Linter (ESLint) + Prettier Extensions

Linter (ESLint):

- A static code analysis tool for identifying and reporting on patterns found in JavaScript code.
- It helps enforce coding standards, identify potential errors, and maintain consistency across codebases.
- ESLint is highly configurable, allowing developers to define custom rules and plugins tailored to their specific project requirements.

Linter (ESLint)

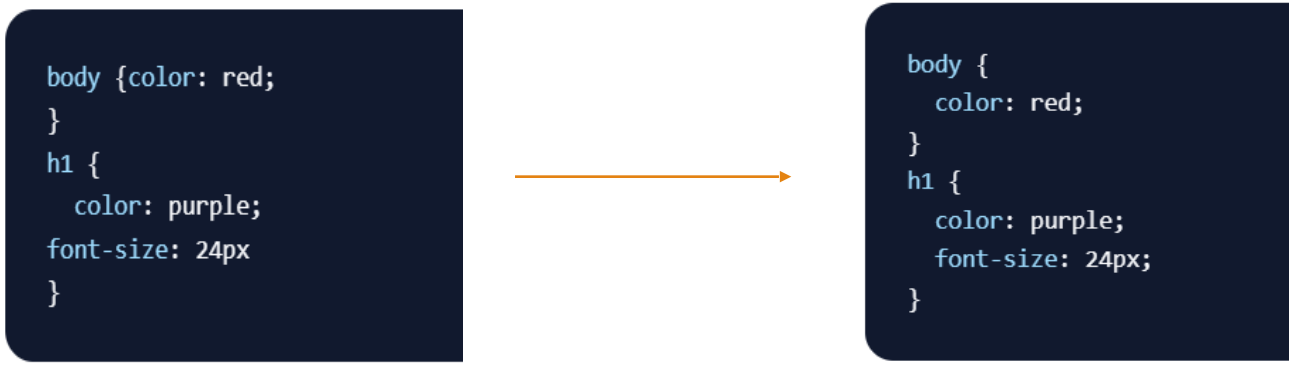
Benefits:

- Ensure that all team members adhere to the same standards, leading to cleaner and more readable code.
- ESLint identifies potential errors and code smells in our codebase during the development phase.
- ESLint promotes the writing of high-quality code. This leads to more maintainable, scalable, and robust applications that are easier to understand, debug, and extend over time.

Linter (ESLint) + Prettier Extensions

Prettier:

- A well-known code formatter that supports a variety of different programming languages. It helps us avoid manually formatting our code by automatically formatting it based on a specified code style.



Local development with Docker

Motivations:

- How do we make sure everyone in the team will have the same development environment?
- How do we make sure code will run in production environment as the same as in local development?

Local development with Docker

Solution:

- Docker !!!
- docker-compose
- Make

Local development with Docker

How:

- Docker image
- Managing environments, packages,… with code
- Local development and production will have the same enviro

  would use the same docker image.

```
FROM node:18-alpine

RUN apk add make

# Install dependencies for NestJS
RUN yarn global add typescript @nestjs/cli
RUN yarn global add concurrently

WORKDIR /app

ADD . .

EXPOSE 3000
```

Local development with Docker

How:

- docker volume to mount host folder to docker containers
- run the docker image with development commands: hot reload on changes

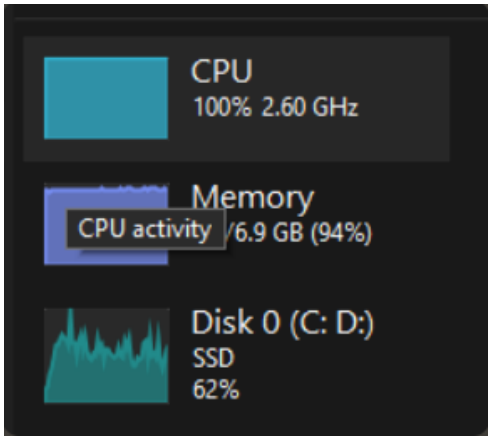
```
volumes:
  # Mount the local directory to the container
  # /app is the working directory in the containers
  - ./:/app
```

```
yarn install
yarn workspace @team8/constants build
yarn workspace @team8/types build
yarn workspace @team8/utils build
concurrently "yarn workspace @team8/frontend build:dev" "yarn workspace @team8/backend start:dev"
```

Local development with Docker

Challenges:

- Not working well with Windows, need to use WSL to enable hot reload.
- Requires a lot of computing resources



Monorepo

Motivations:

- Typescript for backend and frontend
- Can we have FE and BE share and reuse codes?
- Example:
  - Data transfer objects (DTOs) for API requests/responses structures
  - Password validation? Same mechanism in BE and FE -> same function

Monorepo

What:

- Wikipedia: a **monorepo** ("mono" meaning 'single' and "repo" being short for 'repository') is a software-development strategy in which the code for a number of projects is stored in the same repository

Monorepo

How:

- Yarn workspace: specified in root package.json
- Each workspace has its own package.json and name

```
{
    "name": "@team8/backend",
```

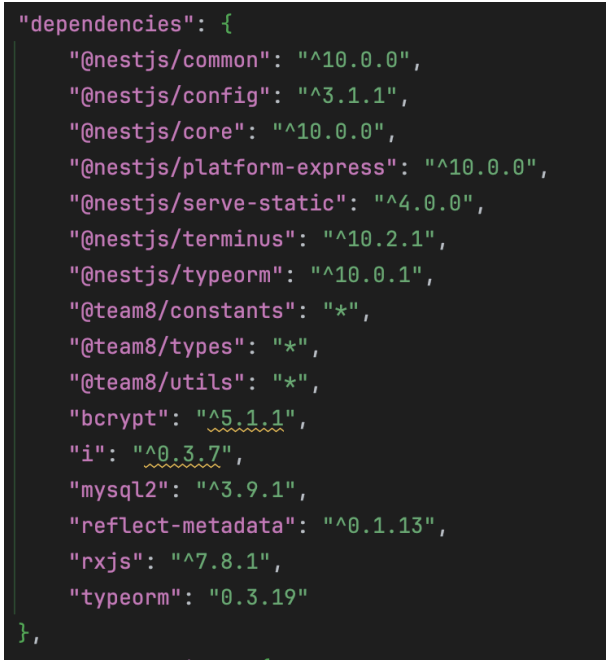
```
"workspaces": {
    "packages": [
        "packages/*",
        "apps/*"
    ],
    "nohoist": [
        "**/jest",
        "**/prisma",
        "**/@prisma/client"
    ]
},
```

Monorepo

How:

- Specify packages in dependencies list inside package.json
- Import and use the shared code

```
import { LogInDto, SignUpDto } from '@team8/types/dtos/auth';
```



Q & A
Thank you!